

Algorithms

Lecture # 36

Syed Hamed Raza

Kruskal's Algorithm

Kruskal's Algorithm

- As the algorithm runs, the edges in A induce a *forest* on the vertices.
- The trees of this forest are eventually merged until a single tree forms containing all vertices.

Kruskal's Algorithm

Kruskal's Algorithm

- The tricky part of the algorithm is how to detect whether the addition of an edge will create a cycle in A .
- Suppose the edge being considered has vertices (u, v) .
- We want a fast test that tells us whether u and v are in the same tree of A .

Kruskal's Algorithm

Kruskal's Algorithm

This can be done using the *Union-Find* data structure which supports the following $O(\log n)$ operations:

Create-set(*u*): Create a set containing a single item *u*.

Find-set(*u*): Find the set that contains *u*.

Union(*u,v*): merge the set containing *u* and set containing *v* into a common set.

Kruskal's Algorithm

Kruskal's Algorithm

- In Kruskal's algorithm, the vertices will be stored in sets.
- The vertices in each tree of A will be a set.
- The edges in A can be stored as a simple list.

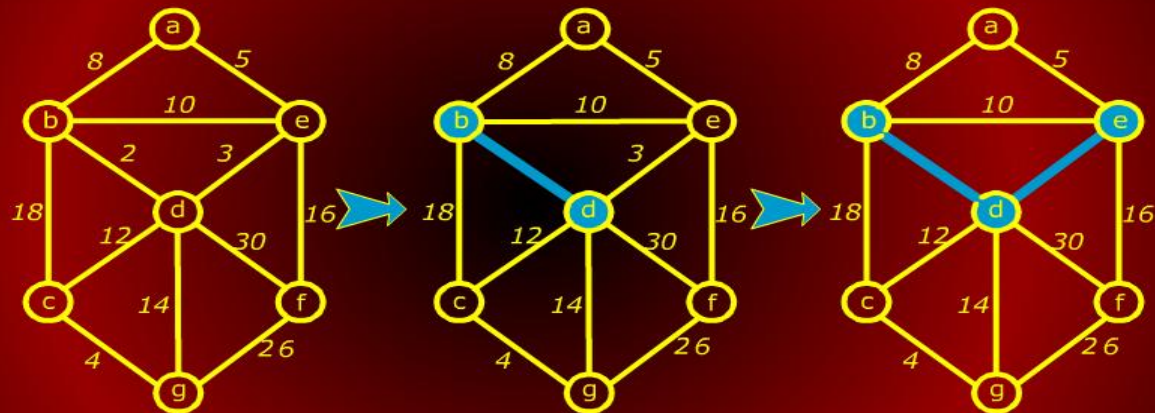
Kruskal's Algorithm

Kruskal's Algorithm

```
KRUSKAL( $G = (V, E)$ )  
1   $A \leftarrow \{\}$   
2  for ( each  $u \in V$ )  
3  do create_set( $u$ )  
4  sort  $E$  in increasing order by weight  $w$   
5  for ( each  $(u, v)$  in sorted edge list)  
6  do if ( $\text{find}(u) \neq \text{find}(v)$ )  
7      then add  $(u, v)$  to  $A$   
8          union( $u, v$ )  
9  return  $A$ 
```

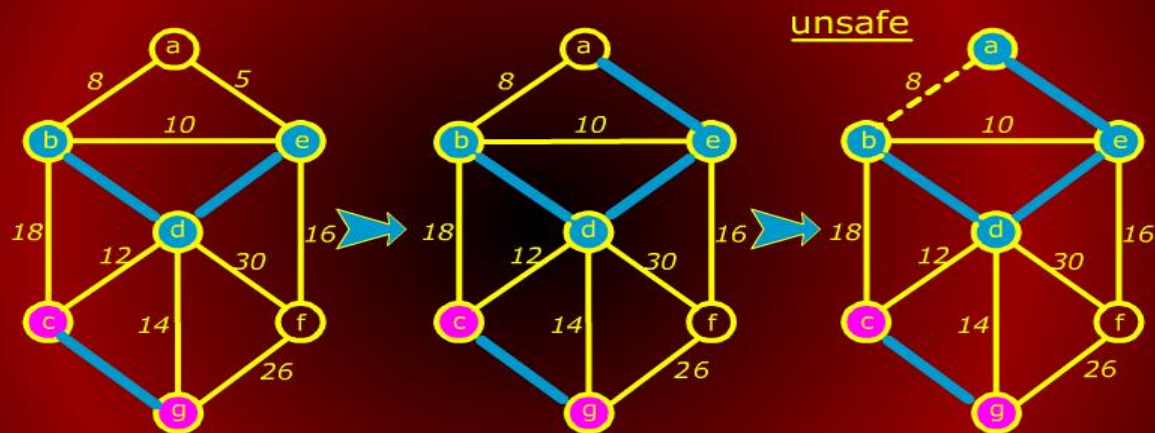

Kruskal's Algorithm Trace

Kruskal's Algorithm Trace



Kruskal's Algorithm Trace

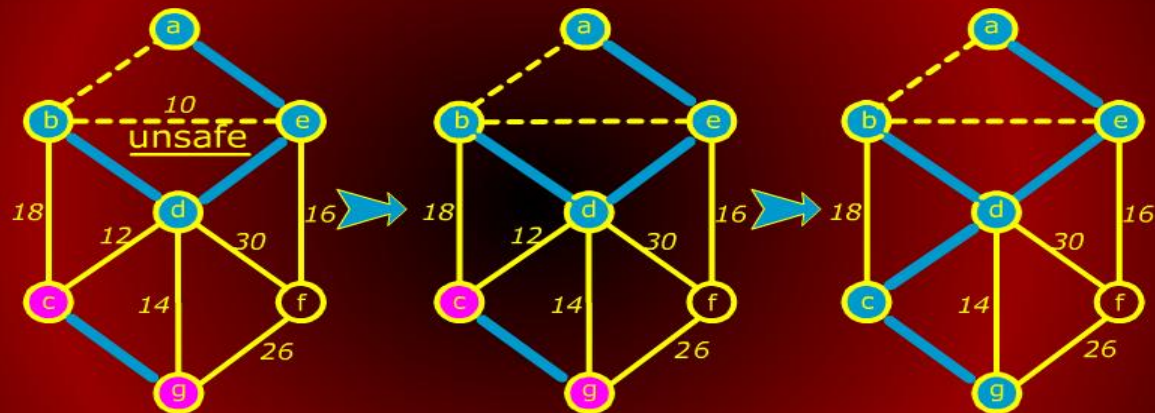
Kruskal's Algorithm Trace



Color indicates union set

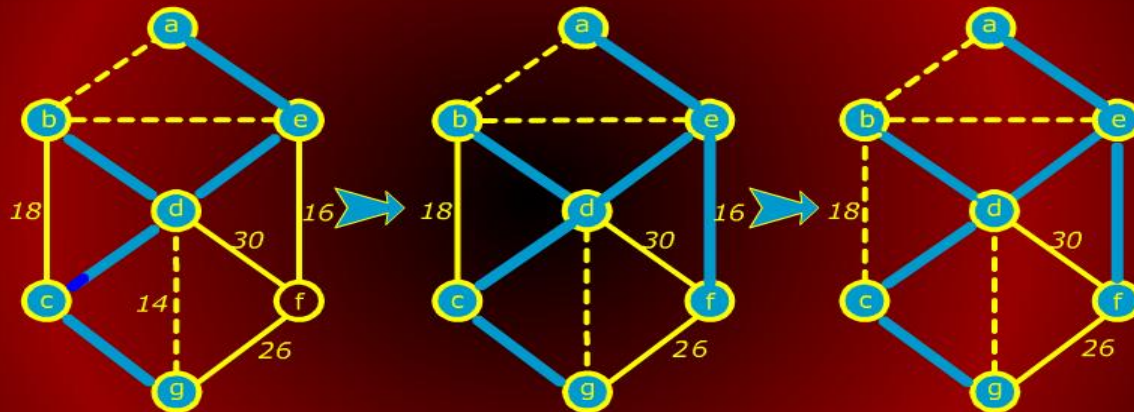
Kruskal's Algorithm Trace

Kruskal's Algorithm Trace



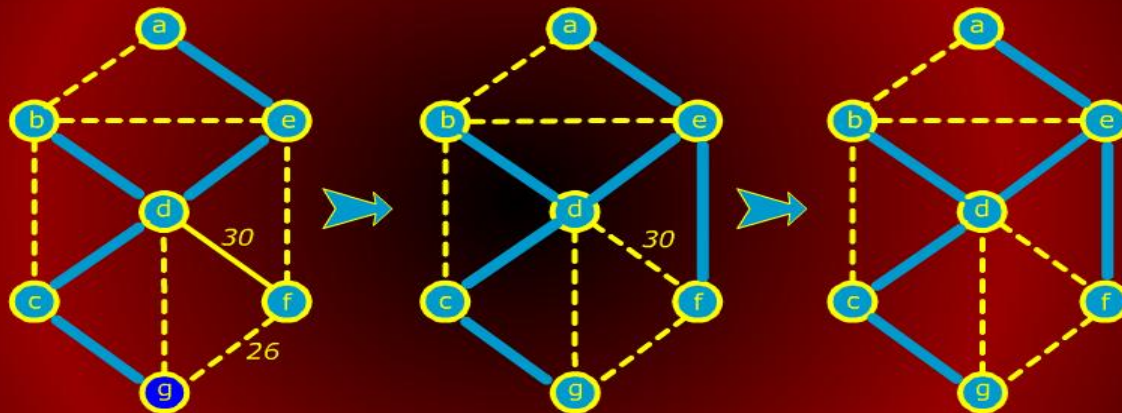
Kruskal's Algorithm Trace

Kruskal's Algorithm Trace



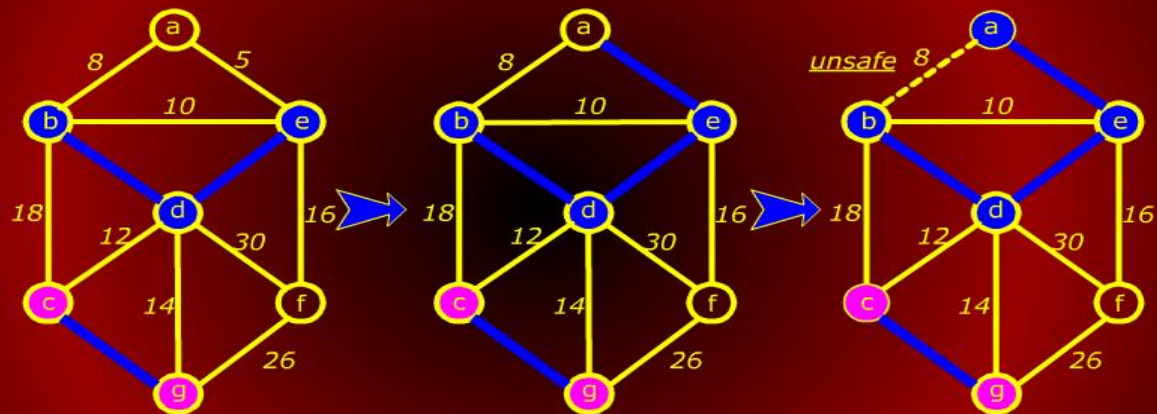
Kruskal's Algorithm Trace

Kruskal's Algorithm Trace



Kruskal's Algorithm Trace

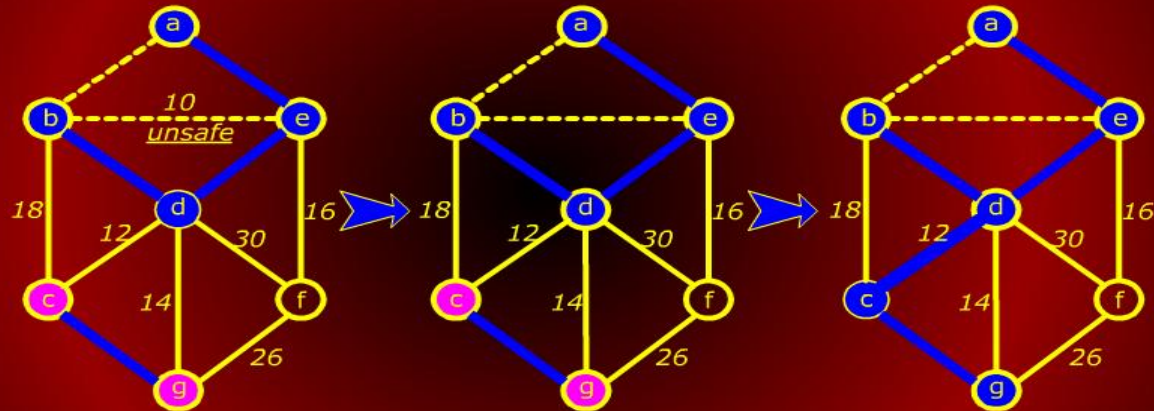
Kruskal's Algorithm Trace



Color indicates union set

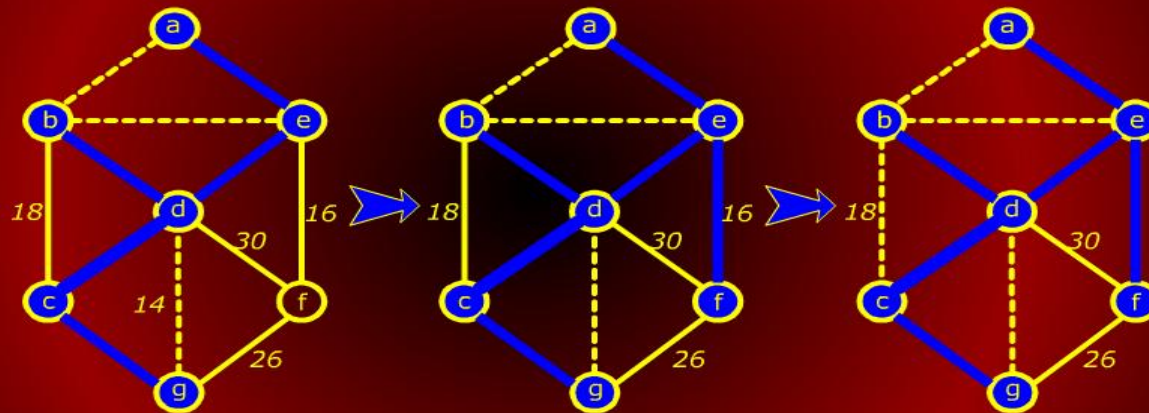
Kruskal's Algorithm Trace

Kruskal's Algorithm Trace



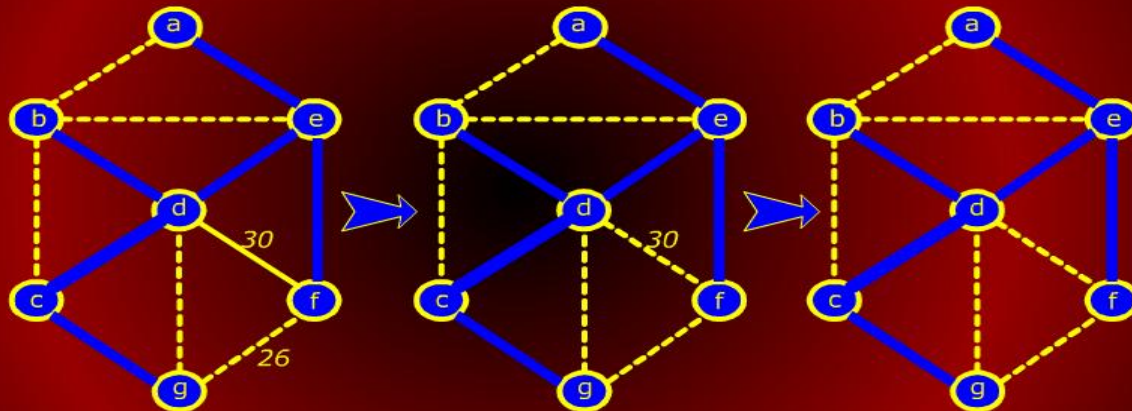
Kruskal's Algorithm Trace

Kruskal's Algorithm Trace



Kruskal's Algorithm Trace

Kruskal's Algorithm Trace



Kruskal's Analysis

Kruskal's Analysis

- Since graph is connected, we may assume that $E \geq V - 1$.
- Sorting edges (4) takes $\Theta(E \log E)$ (4)
- The for loop (5) performs $O(E)$ find and $O(V)$ union operations.

```
KRUSKAL( $G = (V, E)$ )
1   $A \leftarrow \{\}$ 
2  for ( each  $u \in V$  )
3  do create_set( $u$ )
4  sort  $E$  in increasing
   order by weight  $w$ 
5  for (each  $(u, v)$  in
   sorted edge list)
6  do if ( $\text{find}(u) \neq \text{find}(v)$ )
7     then add  $(u, v)$  to  $A$ 
8         union( $u, v$ )
9  return  $A$ 
```

Kruskal's Analysis

Kruskal's Analysis

- Total time for union - find is $O(E\alpha(V))$ where $\alpha(V)$ is the inverse Ackerman function.
- $\alpha(V) < 4$ for V less the number of atoms in the entire universe.
- Thus the time is dominated by sorting.
- Overall time for Kruskal is $\Theta(E \log E) = \Theta(E \log V)$ if the graph is sparse.

Prim's Algorithm

Prim's Algorithm

- Prim's algorithm builds the MST by adding leaves one at a time to the current tree.
- We start with a root vertex r ; it can be any vertex.
- At any time, the subset of edges A forms a single tree (in Kruskal's, it formed a forest).

Minimum Spanning Trees

Minimum Spanning Trees

Kruskal's Algorithm

Kruskal's Algorithm

- Kruskal's algorithm works by adding edges in increasing order of weight (lightest edge first).
- If the next edge does not induce a cycle among the current set of edges, then it is added to A .
- If it does, we skip it and consider the next in order.

Kruskal's Algorithm

Kruskal's Algorithm

- In Kruskal's algorithm, the vertices will be stored in sets.
- The vertices in each tree of A will be a set.
- The edges in A can be stored as a simple list.
- Here is the algorithm: